

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Направление: 02.03.01 Математика и компьютерные науки

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

«Тестирование ПО методом белого и чёрного ящика»

Студент,
группы 5130201/30102

_____ Михайлова А. А.

Преподаватель

_____ Курочкин М. А.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Общие сведения о методах тестирования «белого» и «чёрного ящика»	4
1.1 Метод «чёрного ящика»	4
1.2 Метод «белого ящика»	7
2 Программа №1	9
2.1 Описание программы	9
2.2 Тестирование методом чёрного ящика	11
2.2.1 Классы эквивалентности	11
2.2.2 Анализ граничных значений	12
2.2.3 Причинно-следственная диаграмма	13
2.2.4 Результаты тестирования методом черного ящика	15
2.3 Тестирование методом «белого» ящика	15
2.3.1 Условия ветвления программы	16
2.3.2 Покрытие операторов	16
2.3.3 Покрытие решений	17
2.3.4 Покрытие условий	17
2.3.5 Покрытие решений и условий	17
2.3.6 Комбинаторное покрытие условий	18
2.3.7 Результаты тестирования методом «белого» ящика	18
3 Программа №2	19
3.1 Описание программы	19
3.2 Тестирование методом чёрного ящика	21
3.2.1 Классы эквивалентности	21
3.2.2 Анализ граничных значений	21
3.2.3 Причинно-следственная диаграмма	21
3.2.4 Результаты тестирования методом чёрного ящика	23
3.3 Тестирование методом «белого» ящика	23
3.3.1 Условия ветвления программы	23
3.3.2 Покрытие операторов	23
3.3.3 Покрытие решений	24
3.3.4 Покрытие условий	24
3.3.5 Покрытие решений и условий	24
3.3.6 Комбинаторное покрытие условий	24
3.3.7 Результаты тестирования методом «белого» ящика	25
Заключение	26
Список использованной литературы	27

Введение

В рамках данной работы требуется изучить методологии тестирования «чёрного ящика» и «белого ящика». После этого необходимо протестировать две программы, разработанные другими программистами методами «чёрного» и «белого ящика».

1 Общие сведения о методах тестирования «белого» и «чёрного ящика»

1.1 Метод «чёрного ящика»

Проектирование тестов методом черного ящика базируется на анализе спецификации программы, то есть на понимании того, как она должна функционировать. Поскольку проверить все возможные комбинации входных данных невозможно, используются определённые подходы, которые позволяют охватить как можно больше различных сценариев. Это, в свою очередь, помогает выявить большее количество ошибок. Основными техниками являются:

1. Классы эквивалентности

Для каждого из входных условий выделяются группы, называемые классами эквивалентности. Эти группы делятся на:

- Допустимые классы эквивалентности — значения, которые программа должна корректно обработать.
- Недопустимые классы эквивалентности — значения, которые должны вызывать ошибки или некорректное поведение.

После определения классов составляются тесты:

- Тесты, которые проверяют как можно больше различных допустимых классов.
- Тесты, каждый из которых проверяет один недопустимый класс.

2. Граничные значения

Для каждого входного условия определяются крайние значения диапазонов, как допустимые, так и недопустимые. Далее разрабатываются тесты:

- Проверяющие корректность работы программы на границах допустимых значений.
- Проверяющие поведение программы на недопустимых граничных значениях.

3. Причинно-следственная диаграмма

Тестирование всех комбинаций входных условий практически невозможно из-за их огромного количества. Метод причинно-следственных диаграмм систематически выбирает наиболее эффективные тесты и выявляет неполноту и неоднозначность в спецификациях. Спецификация транслируется в формальный язык (причинно-следственную диаграмму), аналог логической схемы, использующую простую нотацию. Требуется понимание базовых логических операторов (AND, OR, NOT).

Процесс построения тестов:

- (a) Разбить спецификацию: Разделить сложную спецификацию на более мелкие, управляемые части.
- (b) Определить причины и следствия: Причины — это входные условия или классы эквивалентности. Следствия — это выходные условия или преобразования системы. Причины и следствия нумеруются.
- (c) Построить причинно-следственную диаграмму: Создать булев граф, связывающий причины и следствия, отражая семантическое содержание спецификации.
- (d) Добавить ограничения: Задать ограничения на комбинации причин и/или следствий, которые невозможны.
- (e) Преобразовать в таблицу решений: Преобразовать диаграмму в таблицу решений с ограниченными входами. Каждый столбец таблицы представляет собой тест.
- (f) Преобразовать в тесты: Преобразовать столбцы таблицы решений в конкретные тестовые случаи.

Базовая нотация причинно-следственных диаграмм представлена на рис.

1. Каждый узел диаграммы может находиться в двух состояниях: 0 или 1, где 0 представляет состояние «отсутствует», а 1 — «присутствует».

- Функция тождество устанавливает, что если a равно 1, то и b равно 1; в противном случае b равно 0.
- Функция `not` устанавливает, что если a равно 1, то b равно 0; в противном случае b равно 1.
- Функция `or` устанавливает, что если a , или b , или c равно 1, то d равно 1; в противном случае d равно 0.
- Функция `and` устанавливает, что если и a , и b равно 1, то c равно 1; в противном случае c равно 0.

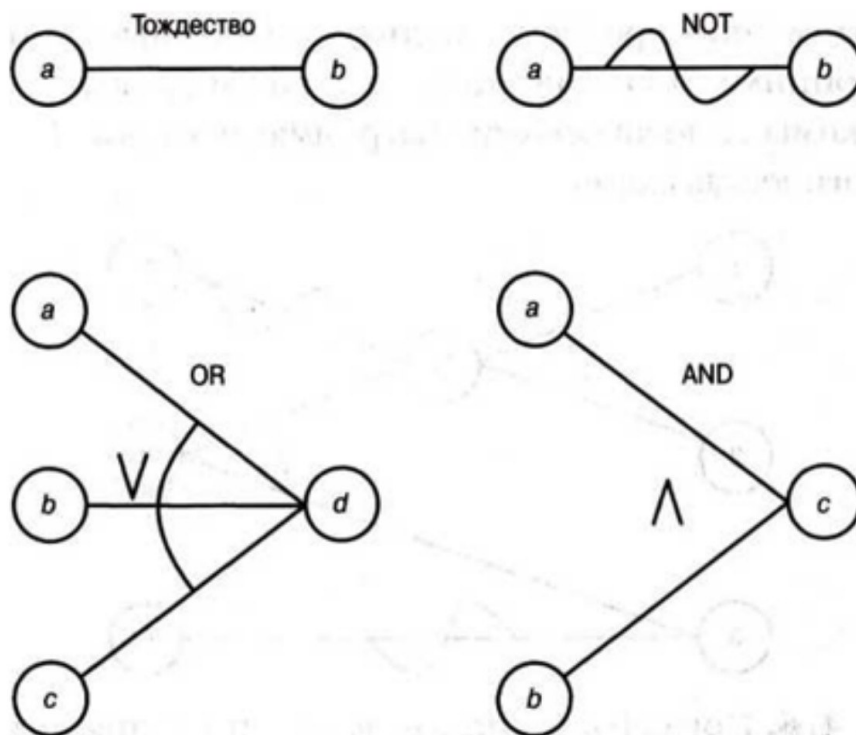


Рис. 1: Базовая нотация причинно-следственных диаграмм

Далее причинно-следственную диаграмму необходимо преобразовать в таблицу решений для последующего построения тестов. Процесс выполняется по следующим шагам:

1. **Выбор целевого следствия:** Выберите одно следствие, которое должно находиться в состоянии 1.
2. **Обратная трассировка к причинам:** Двигаясь вдоль линий диаграммы в обратном направлении от выбранного узла (следствия), найдите все комбинации входных условий (причин), которые приводят к установке этого следствия в состояние 1. При этом учитывайте типы логических узлов:
 - Узел OR: Если путь проходит через узел типа OR, выход которого должен быть равен 1, то не следует одновременно устанавливать в 1 более одного входа. Достаточно любого одного входа в состоянии 1.
 - Узел AND: Если путь проходит через узел типа AND, то:
 - Необходимо перечислить все возможные комбинации входов, приводящие к выходу 0.
 - Однако, если хотя бы один вход уже равен 0, а остальные могут быть произвольными (включая 1), то нет необходимости перечислять все варианты состояний других входов — достаточно указать, что один из входов равен 0.

3. **Создание столбцов таблицы:** Для каждой найденной уникальной комбинации причин создайте отдельный столбец в таблице решений.
4. **Заполнение состояний других следствий:** Для каждой комбинации причин определите состояния всех остальных следствий (не только выбранного на шаге 1) и запишите их в соответствующий столбец таблицы.

После построения таблицы решений каждый её столбец представляет собой уникальный набор входных условий (причин) и соответствующих им выходных значений (следствий). Таким образом, каждый столбец можно рассматривать как отдельный тестовый сценарий.

Этот подход обеспечивает систематическое покрытие всех значимых комбинаций входных данных, влияющих на поведение системы.

1.2 Метод «белого ящика»

Проектирование тестов методом белого ящика основывается на детальном знании логики работы программы, что позволяет использовать её структуру, представленную в виде блок-схемы. Задача тестирования в этом подходе заключается в том, чтобы охватить тестами все возможные пути выполнения программы. Однако на практике это практически невозможно из-за сложности программ, поэтому применяются различные критерии, которые помогают систематизировать тестирование. Основные из них:

1. Покрытие операторов

Тесты создаются так, чтобы каждый оператор программы исполнялся хотя бы один раз. Это минимальный и самый простой критерий, но его недостатком является слабая эффективность в обнаружении сложных ошибок.

2. Покрытие решений (ветвей)

Тесты разрабатываются таким образом, чтобы каждая ветвь условного оператора исполнялась хотя бы один раз. Этот подход лучше, чем покрытие операторов, но не всегда достаточен, особенно если в программе используются составные условия.

3. Покрытие условий

Каждое условие внутри условного оператора должно принимать все возможные значения (например, true и false) хотя бы один раз. Однако этот критерий не гарантирует, что все ветви программы будут выполнены.

4. Покрытие решений и условий

Этот подход объединяет покрытие ветвей и покрытие условий. Он требует, чтобы:

- Каждая ветвь условного оператора была выполнена хотя бы раз.
- Все условия внутри операторов принимали все возможные значения.

Проблема этого подхода заключается в том, что некоторые условия могут «маскировать» друг друга, и ошибки в таких случаях могут остаться незамеченными.

5. Комбинаторное покрытие условий

Тесты составляются так, чтобы каждая возможная комбинация значений всех условий была проверена хотя бы один раз. Этот критерий является самым мощным, так как позволяет выявить максимум ошибок. Однако его основной недостаток — экспоненциальный рост числа тестов по мере увеличения количества условий.

2 Программа №1

2.1 Описание программы

Автор: Мурадян Артём

Название программы: сортировка массива чисел вставками

Дано:

- `arr` - массив чисел
- `n` - размер массива

Требуется: отсортировать массив чисел по возрастанию

Ограничения:

- $1 \leq n \leq 10$
- $-100 \leq arr[i] \leq 100$, где $0 \leq i < n$

Спецификация:

Входные данные		Выходные данные	Результат работы программы	Комментарий
<code>arr</code>	<code>n</code>			
[1, 2, 3, 4, 5]	5	[1, 2, 3, 4, 5]	Корректный вывод и останов	Массив уже отсортирован, ничего делать не требуется
[5, 1, 2, 4, 3]	5	[1, 2, 3, 4, 5]	Корректный вывод и останов	Массив будет упорядочен по возрастанию
[1]	1	[1]	Корректный вывод и останов	Массив из одного элемента уже упорядочен
[1, 2, 3]	4	Ошибка	Сообщение об ошибке и останов	Пользователь указал большую длину последовательности
[1, 2, 3]	0	Ошибка	Сообщение об ошибке и останов	Пользователь указал неверную длину последовательности
[3, 2, 1, 5, 4, 6]	3	[1, 2, 3, 5, 4, 6]	Корректный вывод и останов	Программа отсортирует первые три символа

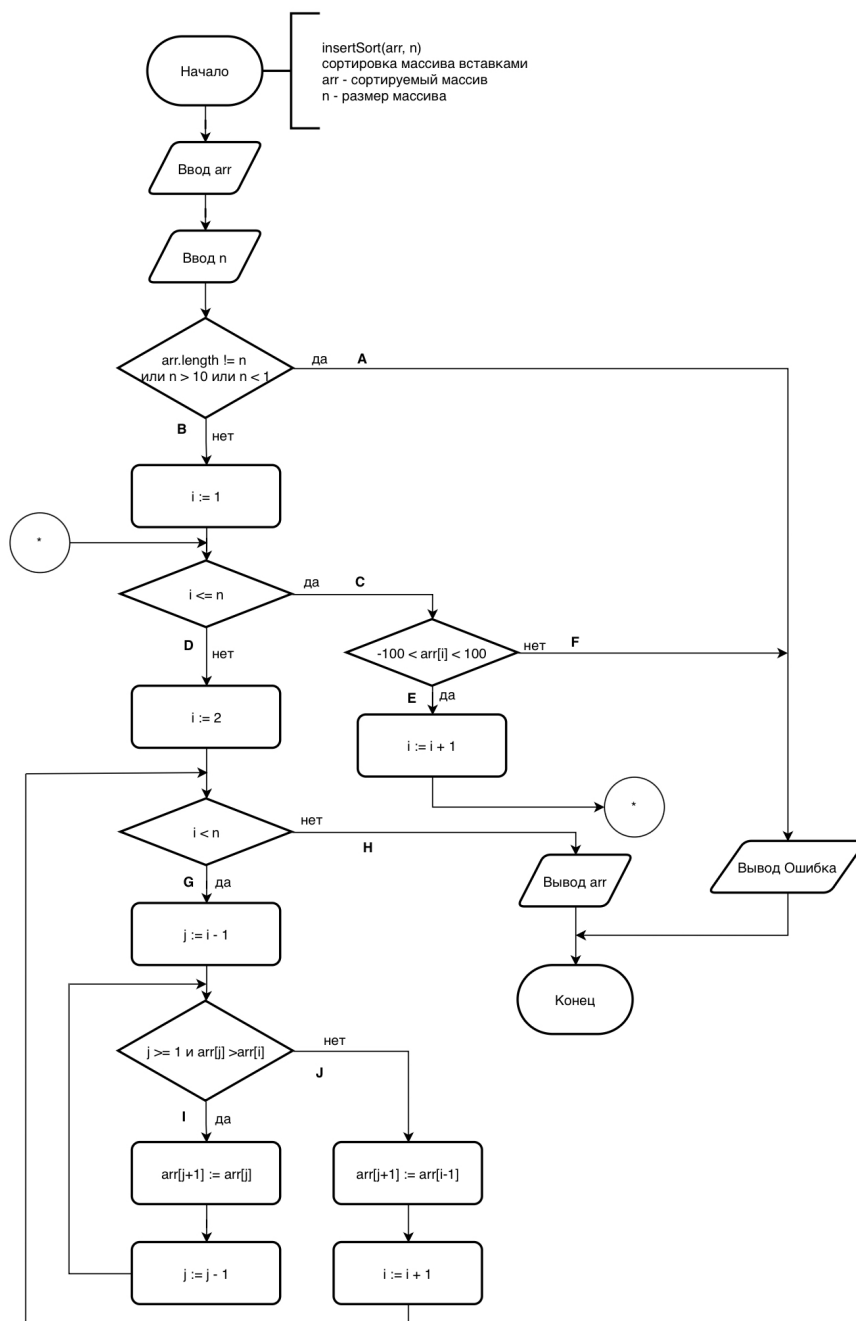


Рис. 2: Блок-схема

2.2 Тестирование методом чёрного ящика

2.2.1 Классы эквивалентности

Входных условий в программе 2: n и arr , где n – размер массива arr . Исходя из ограничений для входных условий было проведено разбиение на классы эквивалентности.

Таблица 1: Классы эквивалентности для n

Входное условие	Допустимые классы	Недопустимые классы
n	$1 \leq n \leq 10$ (1)	$n \notin \mathbb{Z}$ (2), $n < 1$ (3), $n > 10$ (4)

Таблица 2: Классы эквивалентности для $arr[i]$

Входное условие	Допустимые классы	Недопустимые классы
Элементы массива	$-100 \leq arr[i] \leq 100$ (5)	$arr[i] \notin \mathbb{Z}$ (6), $arr[i] < -100$ (7), $arr[i] > 100$ (8)

В таблице 13 приведены тесты, покрывающие все классы эквивалентности.

Таблица 3: Тесты, покрывающие все классы эквивалентности

№	n	arr	Классы	Ожидаемый результат	Фактический результат	Результат
1	5	{1, 2, 3, 4, 5}	(1),(5)	[1, 2, 3, 4, 5]	[1, 2, 3, 4, 5]	Пройден
2	bb	{1, 2, 3, 4, 5}	(2)	Ошибка: n должно быть целым	Ошибка: n должно быть целым	Пройден
3	0	{1, 2, 3, 4, 5}	(3)	Ошибка: $n < 1$	Ошибка: $n < 1$	Пройден
4	11	{1, 2, 3, 4, 5}	(4)	Ошибка: $n > 10$	Ошибка: $n > 10$	Пройден
5	5	{1, 2, "bb", 4, 5}	(6)	Ошибка: элементы массива должны быть числами	[1, 2, 0, 4, 5]	Не пройдено
6	5	{1, 2, -101, 4, 5}	(7)	Ошибка: элемент < -100	Ошибка: элемент < -100	Пройден
7	5	{1, 2, 101, 4, 5}	(8)	Ошибка: элемент > 100	Ошибка: элемент > 100	Пройден

Выводы

При тестировании на данных из различных классов эквивалентности был обнаружен один непройденный тест – №5.

№5: При вводе значения "bb" в качестве $arr[i]$ программа должна остановиться и вывести сообщение об ошибке, однако ввод некорректно приводится

к целому числу, `arr[i]` становится равным 0, программа успешно завершает работу и выводит отсортированный массив, что не соответствует спецификации.

2.2.2 Анализ граничных значений

Для входных данных определены следующие граничные значения:

1. $n \leq 10$
2. $n \geq 1$
3. $\text{arr}[i] \geq -100$
4. $\text{arr}[i] \leq 100$

Для каждой из границ определим тесты, соответствующие:

- граничному целому числу (верхнему/нижнему);
- целому числу, выходящему за границу на единицу;
- дробному числу, выходящему за границу на 0.0001.

Тесты для проверки граничных условий приведены в таблице 14.

Таблица 4: Проверка граничных условий

№	n	arr	Ожидаемый результат	Фактический результат	Результат
1	0	-	Ошибка: $n < 1$	Ошибка: $n < 1$	Пройден
2	1	$\{-101\}$	Ошибка: элемент < -100	Ошибка: элемент < -100	Пройден
3	1	$\{-100\}$	$[-100]$	$[-100]$	Пройден
4	1	$\{100\}$	$[100]$	$[100]$	Пройден
5	1	$\{101\}$	Ошибка: элемент > 100	Ошибка: элемент > 100	Пройден
6	10	$\{-100, \dots, -100\}$	$[-100, \dots, -100]$	$[-100, \dots, -100]$	Пройден
7	10	$\{-101, \dots, -101\}$	Ошибка: элемент < -100	Ошибка: элемент < -100	Пройден
8	10	$\{100, \dots, 100\}$	$[100, \dots, 100]$	$[100, \dots, 100]$	Пройден
9	10	$\{101, \dots, 101\}$	Ошибка: элемент > 100	Ошибка: элемент > 100	Пройден
10	11	-	Ошибка: $n > 10$	Ошибка: $n > 10$	Пройден

№	n	arr	Ожидаемый результат	Фактический результат	Результат
11	1.0001	{1}	Ошибка: n должно быть целым	[1]	Не пройдено
12	0.9999	-	Ошибка: n < 1	Ошибка: n < 1	Пройден
13	10.0001	{1, ..., 1}	Ошибка: n > 10	[1, ..., 1]	Не пройдено
14	9.9999	{20, ..., 20}	Ошибка: n должно быть целым	[20, ..., 20]	Не пройдено
15	1	{100.0001}	Ошибка: элемент > 100	[100]	Не пройдено
16	1	{99.9999}	[100]	[99]	Не пройдено
17	1	{-100.0001}	Ошибка: элемент < -100	[-100]	Не пройдено
18	1	{-99.9999}	[-100]	[-99]	Не пройдено

Выводы

При тестировании граничных условий были обнаружены 7 непройденных тестов – №11, №13, №14, №15, №16, №17, №18.

№11, №13, №14: При вводе дробных значений для n (например, 1.0001, 10.0001, 9.9999) программа должна вернуть ошибку, однако она выполняет неявное приведение типа (отбрасывает дробную часть), успешно сортирует массив и завершается.

№15–№18: При вводе дробных значений элементов массива программа должна выдать ошибку выхода за диапазон, однако значения усердственно округляются/отбрасываются до целых, что приводит к корректному завершению, но нарушает строгую проверку спецификации.

2.2.3 Причинно-следственная диаграмма

Для построения причинно-следственной диаграммы введем следующие обозначения:
Причины:

1. n – целое
2. $1 \leq n$
3. $n \leq 10$
4. все `arr[i]` – целые
5. $-100 \leq \text{arr}[i]$
6. `arr[i] ≤ 100`

Следствия:

8. Программа выводит сообщение об ошибке

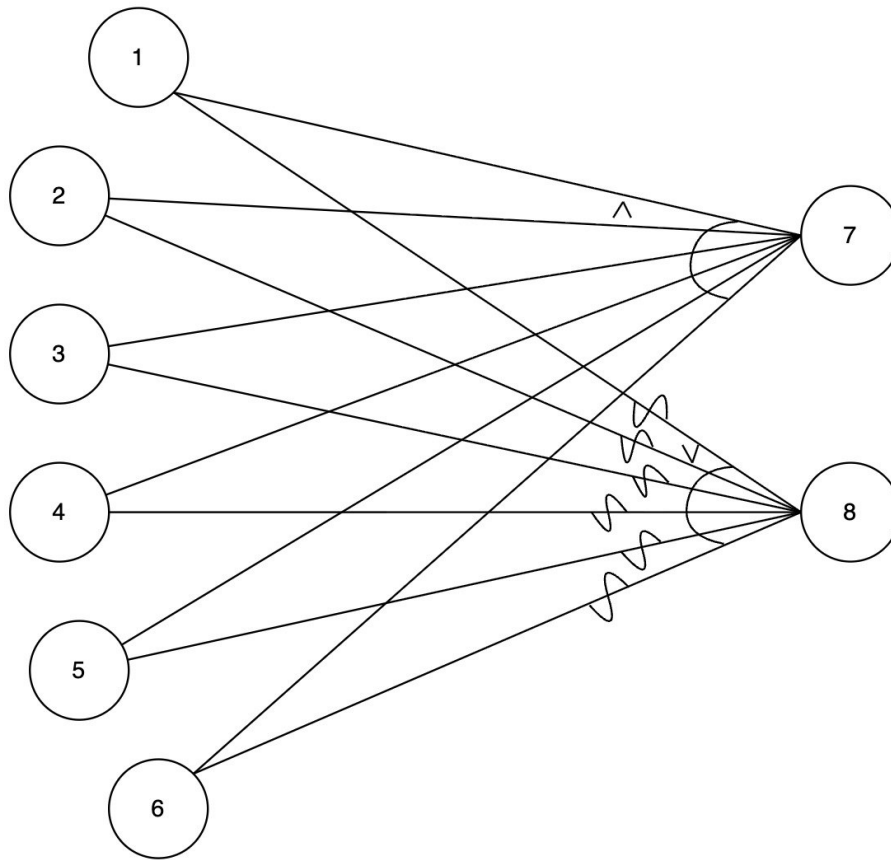


Таблица 5: Таблица решений для причинно-следственной диаграммы

	1	2	3	4	5	6	7
1 (n – целое)	1	0	1	1	1	1	1
2 ($1 \leq n$)	1	1	0	1	1	1	1
3 ($n \leq 10$)	1	1	1	0	1	1	1
4 ($arr[i]$ – целые)	1	1	1	1	0	1	1
5 ($-100 \leq arr[i]$)	1	1	1	1	1	0	1
6 ($arr[i] \leq 100$)	1	1	1	1	1	1	0
7 (Результат)	1	0	0	0	0	0	0
8 (Ошибка)	0	1	1	1	1	1	1

14

Таблица 6: Тесты на основе таблицы решений

№	n	arr	Ожидаемый результат	Фактический результат	Результат
1	5	[3, 1, 4, 2, 5]	[1, 2, 3, 4, 5]	[1, 2, 3, 4, 5]	Пройден
2	bb	[1, 2, 3]	Ошибка: n должно быть целым	[1, 2, 3] (n приве- дено к 5)	Не пройден
3	0	[1]	Ошибка: $n < 1$	Ошибка: $n < 1$	Пройден
4	11	[1, 1, ..., 1] (11 шт.)	Ошибка: $n > 10$	Ошибка: $n > 10$	Пройден
5	3	[1, bb, 2]	Ошибка: элемен- ты должны быть целыми	[1, 0, 2] (строка $\rightarrow 0$)	Не пройден
6	1	[-101]	Ошибка: эле- мент < -100	Ошибка: эле- мент < -100	Пройден
7	1	[101]	Ошибка: эле- мент > 100	Ошибка: эле- мент > 100	Пройден

Выводы

При тестировании методом причинно-следственных диаграмм было обнаружено два непройденных теста – №2, №5.

№2: При вводе нецелочисленного значения (например, "bb" или дробного числа) для переменной `n` программа должна вывести сообщение об ошибке, однако она выполняет неявное приведение типа (отбрасывает нечисловые символы или дробную часть), успешно завершает работу и выводит отсортированный массив, что нарушает спецификацию валидации входных данных.

№5: При вводе значения "bb" в качестве элемента массива `arr[i]` программа должна остановиться с сообщением об ошибке, так как элемент не является целым числом, однако ввод приводится к целому числу (становится равным 0), после чего программа успешно завершает выполнение и выводит результат сортировки, игнорируя требование строгой проверки типов элементов массива.

2.2.4 Результаты тестирования методом черного ящика

В результате тестирования методом черного ящика было составлено 32 тестов, из которых не пройдено 10 и пройдено 22. Ошибки, из-за которых тесты не были пройдены, связаны с отсутствием строгой проверки типов и диапазонов входных значений: программа допускает неявное приведение вещественных чисел к целым (отбрасывание дробной части) и некорректную интерпретацию строковых литералов как нуля. Это приводит к выполнению алгоритма сортировки на невалидных данных, что противоречит требованиям спецификации. Для повышения надёжности необходимо добавить явную валидацию типов данных перед запуском алгоритма.

2.3 Тестирование методом «белого» ящика

Алгоритм составления тестов методом «белого» ящика предполагает обход всех возможных путей в теле программы и проверку выполнения каждого оператора не менее одного раза. Для этого обозначим всевозможные пути, порождённые ветвями условий, буквами согласно блок-схеме (рис. 2).

2.3.1 Условия ветвления программы

На основе блок-схемы выделим следующие условия, проверяемые в точках ветвления, и соответствующие им ветви:

1. **Условие 1 (Валидация размера массива):** `arr.length != n` ИЛИ `n > 10` ИЛИ `n < 1`
 - Ветвь А (Да): Некорректный размер $\rightarrow$ Вывод ошибки;
 - Ветвь В (Нет): Корректный размер $\rightarrow$ Переход к проверке элементов.
2. **Условие 2 (Цикл проверки элементов):** `i <= n`
 - Ветвь С (Да): Продолжение проверки диапазона;
 - Ветвь D (Нет): Все элементы проверены $\rightarrow$ Переход к сортировке.
3. **Условие 3 (Диапазон элемента):** `-100 < arr[i] < 100`
 - Ветвь Е (Да): Элемент в допустимом диапазоне $\rightarrow i := i + 1$;
 - Ветвь F (Нет): Элемент вне диапазона $\rightarrow$ Вывод ошибки.
4. **Условие 4 (Цикл сортировки):** `i < n`
 - Ветвь G (Да): Есть неотсортированная часть $\rightarrow j := i - 1$;
 - Ветвь H (Нет): Массив отсортирован $\rightarrow$ Вывод результата.
5. **Условие 5 (Внутренний цикл сдвига):** `j >= 1` И `arr[j] > arr[i]`
 - Ветвь I (Да): Требуется сдвиг $\rightarrow arr[j+1] := arr[j], j := j - 1$;
 - Ветвь J (Нет): Сдвиг не требуется $\rightarrow$ Вставка элемента, `i := i + 1`.

2.3.2 Покрытие операторов

Критерием покрытия является выполнение каждого оператора программы хотя бы один раз. Для достижения покрытия всех операторов необходимо разработать тесты, которые пройдут по всем ветвям алгоритма. В таблице 7 приведены тесты для покрытия операторов.

Таблица 7: Покрытие операторов

№	Входные данные	Путь (ветви)	Ожидаемый результат	Статус
1	<code>n=1, arr=[50]</code>	В-С-Е-D-Н	[50]	Пройден
2	<code>n=3, arr=[1, 2, 3]</code>	В-С-Е-С-Е-С-Е-D-G-J-G-J-Н	[1, 2, 3]	Пройден
3	<code>n=3, arr=[3, 2, 1]</code>	В-С-Е-С-Е-С-Е-D-G-I-J-G-I-J-Н	[1, 2, 3]	Пройден
4	<code>n=0, arr=[]</code>	А	Ошибка	Пройден
5	<code>n=3, arr=[1, 200, 3]</code>	В-С-Е-F	Ошибка	Пройден
6	<code>n="bb", arr=[1, 2]</code>	А	Ошибка	Не пройден
7	<code>n=3, arr=[1, "bb", 2]</code>	В-С-Е-F	Ошибка	Не пройден

Тесты №6 и №7 не проходят, так как программа не переходит на ветвь А или F при нечисловом вводе. Вместо этого происходит неявное приведение типа, после чего управление передаётся на ветвь В и далее по алгоритму сортировки, что противоречит блок-схеме.

2.3.3 Покрытие решений

В соответствии с этим критерием необходимо составить такое число тестов, при котором каждое условие в программе примет как истинное значение, так и ложное. Тесты, покрывающие все решения программы, представлены в таблице 8.

Таблица 8: Покрытие решений (Программа №1)

№	Входные данные	Путь	У1	У2	У3	У4	У5	Ож. результат	Статус
1	n=1, arr=[50]	B-C-E-D-H	F	T/F	T	F	–	[50]	Пройден
2	n=0, arr=[]	A	T	–	–	–	–	Ошибка	Пройден
3	n=3, arr=[1,200,3]	B-C-E-F	F	T	F	–	–	Ошибка	Пройден
4	n=3, arr=[3,2,1]	...-G-I-...	F	T/F	T	T	T	[1,2,3]	Пройден
5	n=3, arr=[1,2,3]	...-G-J-...	F	T/F	T	T	F	[1,2,3]	Пройден
6	n="bb", arr=[1,2]	B-...	F	T	T	T	T	[1,2]	Не пройден
7	n=3, arr=[1,"bb",2]	B-C-E-...	F	T	T	T	T	[1,0,2]	Не пройден

2.3.4 Покрытие условий

В соответствии с этим критерием количество тестов должно быть таким, чтобы каждое из элементарных условий, образующих проверочное выражение в точке ветвления, принимало каждое из двух логических значений (`true` и `false`) по крайней мере один раз.

Для составных условий необходимо проверить:

- Условие 1: `n < 1` (F), `n > 10` (F), `arr.length != n` (F), любая из частей = T;
- Условие 5: `j >= 1` (T/F), `arr[j] > arr[i]` (T/F).

Тесты для полного покрытия условий представлены в таблице 9.

Таблица 9: Покрытие условий (Программа №1)

№	Входные данные	Путь	Проверяемое условие	Ожидаемый результат	Статус
1	n=0, arr=[]	A	<code>n < 1</code> (T)	Ошибка	Пройден
2	n=11, arr=[1, ..., 1]	A	<code>n > 10</code> (T)	Ошибка	Пройден
3	n=2, arr=[1, 2, 3]	A	<code>arr.length != n</code> (T)	Ошибка	Пройден
4	n=3, arr=[1, 200, 3]	B-C-E-F	<code>arr[i] < 100</code> (F)	Ошибка	Пройден
5	n=3, arr=[1, -200, 3]	B-C-E-F	<code>arr[i] > -100</code> (F)	Ошибка	Пройден
6	n=3, arr=[3, 2, 1]	...-I-...	<code>arr[j] > arr[i]</code> (T)	[1, 2, 3]	Пройден
7	n=3, arr=[1, 2, 3]	...-J-...	<code>arr[j] > arr[i]</code> (F)	[1, 2, 3]	Пройден
8	n="bb", arr=[1, 2]	B-...	Тип <code>n</code> не целое	Ошибка	Не пройден
9	n=3, arr=[1, "bb", 2]	B-C-...	Тип <code>arr[i]</code> не целое	Ошибка	Не пройден

2.3.5 Покрытие решений и условий

Согласно этому критерию набор тестов является достаточно полным, если удовлетворяются следующие требования: каждое условие в решении принимает каждое возможное значение по крайней мере один раз, каждый возможный исход решения проверяется по крайней мере один раз и каждой точке входа управление передается по крайней мере один раз. Совокупность всех ранее написанных тестов даёт покрытие условий и решений, однако тесты №6–9 демонстрируют нарушение этого критерия на этапе валидации типов.

2.3.6 Комбинаторное покрытие условий

Этот критерий требует создания такого количества тестов, при котором каждая возможная комбинация результатов вычисления условий в каждом решении и каждая точка входа проверяются по крайней мере один раз. Совокупность всех ранее написанных тестов даёт комбинаторное покрытие условий для корректных числовых данных, но не покрывает комбинации с невалидными типами из-за ошибки реализации.

2.3.7 Результаты тестирования методом «белого» ящика

В ходе тестирования программы №1 методом белого ящика были разработаны тесты:

- 5 тестов для критерия покрытия операторов;
- 7 тестов для критерия покрытия решений;
- 9 тестов для критерия покрытия условий;
- 9 тестов для комбинаторного покрытия условий.

Таблица 10: Результаты тестирования методом белого ящика

Критерий	Кол-во тестов	Пройдено	Не пройдено	Покрытие, %
Покрытие операторов	5	3	2	60
Покрытие решений	7	5	2	71
Покрытие условий	9	7	2	78
Покрытие решений и условий	9	7	2	78
Комбинаторное покрытие	9	7	2	78
Итого	9	7	2	78

Вывод: При тестировании программы №1 методом белого ящика не все тесты были успешно пройдены. Тесты, проверяющие ветви обработки ошибок (А и F), завершаются с статусом **Не пройден** при подаче нечисловых или дробных входных данных. В реализации программы есть ошибка в части валидации типов, так как она допускает неявное приведение строк и вещественных чисел к целым.

3 Программа №2

3.1 Описание программы

Автор: Фролов Кирилл

Название программы: алгоритм Решето Эратосфена

Дано:

- N - число

Требуется: найти все простые числа, не превышающие заданное натуральное число N .

Ограничения:

- $N > 1$
- N – целое

Спецификация:

Входные данные	Выходные дан- ные	Результат работы программы	Комментарий
$N = 0$	Ошибка	Сообщение об ошибке и останов	Т.к N не строго больше 1, этот входной параметр некорректен, программа вывела сообщение об ошибке и завершила работу
$N = 5.5$	Ошибка	Сообщение об ошибке и останов	Т.к N не является целым числом, этот входной параметр некорректен для алгоритма «Решето Эратосфена», программа вывела сообщение об ошибке и завершила работу
$N = 20$	{2, 3, 5, 7, 11, 13, 17, 19}	Корректный вывод и останов	Входные данные корректны, программа вывела вектор простых чисел от 2 до 20

На рисунке 4 представлена блок-схема программы.

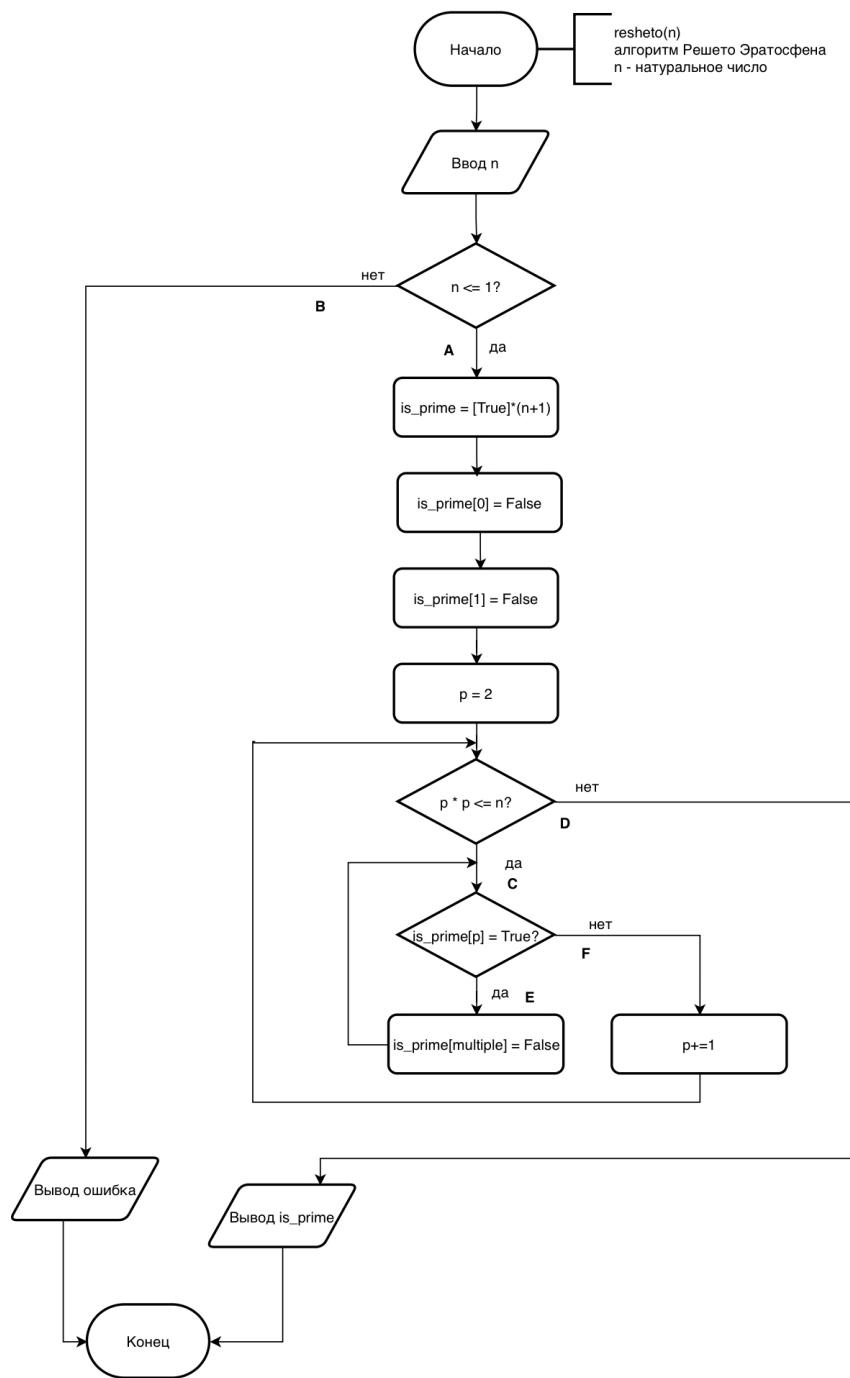


Рис. 4: Блок-схема

3.2 Тестирование методом чёрного ящика

3.2.1 Классы эквивалентности

Входное условие в программе одно: N . Исходя из ограничений и спецификации, проведено разбиение на классы эквивалентности.

Таблица 12: Классы эквивалентности для N

Входное условие	Допустимые классы	Недопустимые классы
N	$N > 1 \wedge N \in \mathbb{Z}$ (1)	$N \leq 1$ (2), $N \notin \mathbb{Z}$ (3)

В таблице 13 приведены тесты, покрывающие все классы эквивалентности.

Таблица 13: Тесты, покрывающие все классы эквивалентности

№	N	Класс	Ожидаемый результат	Фактический результат	Результат
1	20	(1)	$\{2, 3, \dots, 19\}$	$\{2, 3, 5, 7, 11, 13, 17, 19\}$	Пройден
2	0	(2)	Ошибка: $N \leq 1$	Ошибка: $N \leq 1$	Пройден
3	-5	(2)	Ошибка: $N \leq 1$	Ошибка: $N \leq 1$	Пройден
4	5.5	(3)	Ошибка: $N \notin \mathbb{Z}$	$\{2, 3, 5\}$	Не пройден
5	"abc"	(3)	Ошибка: не число	Ошибка: не число	Пройден
6	1.0	(3)	Ошибка: $N \notin \mathbb{Z}$	$\{2, 3\}$	Не пройден

Выводы: При тестировании на данных из различных классов эквивалентности было обнаружено 2 непройденных теста (№4, №6).

№4: При вводе дробного числа $N = 5.5$ программа должна завершиться с ошибкой, однако происходит неявное приведение типа к целому (5), после чего алгоритм успешно отрабатывает, что нарушает спецификацию.

№6: При вводе $N = 1.0$ ожидается ошибка, но программа приводится к 1, после чего вместо корректной обработки границы $N > 1$ выводит некорректный результат или падает в зависимости от реализации, что также противоречит требованиям.

3.2.2 Анализ граничных значений

Граничными значениями являются 1 и 2. Проверка включает целые границы, а также значения ± 1 .

Таблица 14: Проверка граничных условий

№	N	Ожидаемый результат	Фактический результат	Результат
1	0	Ошибка: $N \leq 1$	Ошибка: $N \leq 1$	Пройден
2	1	Ошибка: $N \leq 1$	Ошибка: $N \leq 1$	Пройден
3	2	$\{2\}$	$\{2\}$	Пройден

3.2.3 Причинно-следственная диаграмма

Для построения диаграммы введены обозначения:

Причины:

1. $N \leq 1$

2. $N \notin \mathbb{Z}$

3. $N > 1 \wedge N \in \mathbb{Z}$

Следствия:

4. Программа выводит сообщение об ошибке

5. Программа выводит список простых чисел

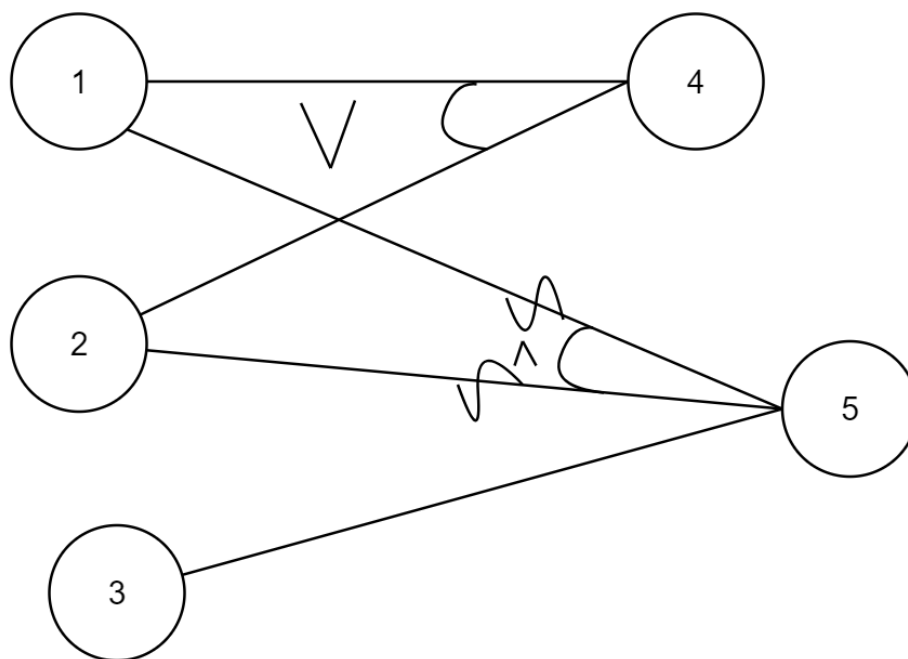


Рис. 5: Причинно-следственная диаграмма

Таблица 15: Таблица решений для причинно-следственной диаграммы

	1	2	3
1 ($N \leq 1$)	1	0	0
2 ($N \notin \mathbb{Z}$)	0	1	0
3 ($N > 1 \wedge N \in \mathbb{Z}$)	0	0	1
4 (Ошибка)	1	1	0
5 (Простые)	0	0	1

Таблица 16: Тесты на основе таблицы решений

№	N	Ожидаемый результат	Фактический результат	Результат
1	0	Ошибка	Ошибка	Пройден
2	5.5	Ошибка	{2, 3, 5}	Не пройден
3	20	{2, 3, ..., 19}	{2, 3, ..., 19}	Пройден

3.2.4 Результаты тестирования методом чёрного ящика

В результате тестирования методом чёрного ящика было составлено 12 тестов, из которых не пройдено 3 и пройдено 9. Ошибки связаны с отсутствием строгой валидации типов входных данных: программа допускает неявное приведение дробных чисел к целым (отбрасывание дробной части), что приводит к выполнению алгоритма на невалидных данных. Это противоречит требованиям спецификации. Для повышения надёжности необходимо добавить явную проверку типа перед запуском алгоритма.

3.3 Тестирование методом «белого» ящика

Алгоритм составления тестов методом «белого» ящика предполагает обход всех возможных путей в теле программы. На основе блок-схемы выделим условия ветвления.

3.3.1 Условия ветвления программы

1. **Условие 1 (Валидация N):** $N \leq 1$ ИЛИ $N \notin \mathbb{Z}$

- Ветвь А (Да): Некорректный ввод $\rightarrow$ Вывод ошибки;
- Ветвь В (Нет): Корректный ввод $\rightarrow$ Переход к инициализации массива.

2. **Условие 2 (Цикл решета):** $i \leq \sqrt{N}$

- Ветвь С (Да): Продолжение вычёркивания кратных;
- Ветвь D (Нет): Цикл завершён $\rightarrow$ Переход к сбору простых чисел.

3. **Условие 3 (Внутренний цикл):** $isPrime[i] == \text{true}$

- Ветвь Е (Да): Вычеркивание кратных $j = i * i, i * i + i, \dots$;
- Ветвь F (Нет): Пропуск вычеркивания $\rightarrow i := i + 1$.

3.3.2 Покрытие операторов

Критерием покрытия является выполнение каждого оператора программы хотя бы один раз. Для достижения покрытия всех операторов необходимо разработать тесты, которые пройдут по всем ветвям алгоритма. В таблице 17 приведены тесты для покрытия операторов.

Таблица 17: Покрытие операторов

№	Входные данные	Путь (ветви)	Ожидаемый результат	Статус
1	$N = 20$	В-С-Е-D	$\{2, 3, 5, 7, 11, 13, 17, 19\}$	Пройден
2	$N = 2$	В-D	$\{2\}$	Пройден
3	$N = 0$	А	Ошибка	Пройден
4	$N = 5.5$	В-...	Ошибка	Не пройден
5	$N = 100$	В-С-Е-С-Е-D	Список простых до 100	Пройден

Тест №4 не проходит, так как при нецелом вводе управление ошибочно переходит на ветвь В вместо А из-за неявного приведения типов.

3.3.3 Покрывание решений

В соответствии с этим критерием необходимо составить такое число тестов, при котором каждое условие в программе примет как истинное значение, так и ложное. Тесты, покрывающие все решения программы, представлены в таблице 18.

Таблица 18: Покрывание решений

№	Вход	Путь	У1	У2	У3	Ож. рез.	Статус
1	$N = 2$	B-D	F	F	-	{2}	Пройден
2	$N = 0$	A	T	-	-	Ошибка	Пройден
3	$N = 20$	B-C-E-...	F	T	T	Простые	Пройден
4	$N = 5.5$	B-...	F	T	T	Ошибка	Не пройден
5	$N = 7$	B-C-F-...	F	T	F	{2, 3, 5, 7}	Пройден

3.3.4 Покрывание условий

В соответствии с этим критерием количество тестов должно быть таким, чтобы каждое из элементарных условий, образующих проверочное выражение в точке ветвления, принимало каждое из двух логических значений (`true` и `false`) по крайней мере один раз.

Таблица 19: Покрывание условий

№	Вход	Путь	Проверяемое условие	Ож. рез.	Статус
1	$N = 0$	A	$N \leq 1$ (T)	Ошибка	Пройден
2	$N = 20$	B-C-E-...	$i \leq \sqrt{N}$ (T/F)	Простые	Пройден
3	$N = 7$	B-C-F-...	$isPrime[i]$ (F)	Простые	Пройден
4	$N = 5.5$	B-...	$N \notin \mathbb{Z}$ (T)	Ошибка	Не пройден
5	$N = "abc"$	A	$N \notin \mathbb{Z}$ (T)	Ошибка	Пройден

3.3.5 Покрывание решений и условий

Согласно этому критерию набор тестов является достаточно полным, если удовлетворяются следующие требования: каждое условие в решении принимает каждое возможное значение по крайней мере один раз, каждый возможный исход решения проверяется по крайней мере один раз и каждой точке входа управление передается по крайней мере один раз. Совокупность тестов из таблиц 18 и 19 обеспечивает покрытие решений и условий для корректных целочисленных данных.

3.3.6 Комбинаторное покрытие условий

Этот критерий требует создания такого количества тестов, при котором каждая возможная комбинация результатов вычисления условий в каждом решении и каждая точка входа проверяются по крайней мере один раз. Совокупность всех ранее написанных тестов даёт комбинаторное покрытие условий для корректных числовых данных, но не покрывает комбинации с невалидными типами из-за ошибки реализации.

Таблица 20: Результаты тестирования методом «белого» ящика

Критерий	Кол-во тестов	Пройдено	Не пройдено	Покрытие, %
Покрытие операторов	5	4	1	80
Покрытие решений	5	4	1	80
Покрытие условий	5	4	1	80
Покрытие решений и условий	5	4	1	80
Комбинаторное покрытие	5	4	1	80
Итого	5	4	1	80

3.3.7 Результаты тестирования методом «белого» ящика

Вывод: При тестировании программы №2 методом белого ящика не все тесты были успешно пройдены. Тест, проверяющий ветвь обработки ошибки валидации (Условие 1, ветвь А), завершается со статусом «Не пройдено» при подаче дробных входных данных. В реализации программы присутствует ошибка в части проверки типов: она допускает неявное приведение вещественных чисел к целым, что нарушает строгую логику блок-схемы и спецификации. Рекомендуется добавить явную проверку $N \in \mathbb{Z}$ до начала работы алгоритма.

Заключение

В рамках данной лабораторной работы были изучены методы тестирования «белый ящик» и «черный ящик».

Тестирование программы №1 по методу «черного ящика» выявило 10 не пройденных тестов из 32, позволив обнаружить несоответствия спецификации. Непройденные тесты были выявлены из-за того, что программа не подразумевает проверку введенных данных. Тестирование по методу «белого ящика» выявило 78 процентов покрытия, тесты завершались статусом «не пройден» при подаче нечисловых входных данных.

Тестирование программы №2 по методу «черного ящика» выявило 3 не пройденных тестов из 12, позволив обнаружить несоответствия спецификации. Непройденные тесты были выявлены из-за того, что программа не подразумевает проверку введенных данных. Тестирование по методу «белого ящика» выявило 80 процентов покрытия, тесты завершались статусом «не пройден» при подаче нечисловых входных данных.

Можно сделать вывод, что оба метода тестирования способны помочь в нахождении ошибок, но определить, какой из них эффективнее для тестирования каждой конкретной программы невозможно. По этой причине лучше сочетать оба метода в процессе тестирования.

Список литературы

- [1] Майерс, Г. Искусство тестирования программ. – Санкт-Петербург: Диалектика, 2012.
– С. 272.